Lecture03: Operator and Data type in JS

That one physical reality splits every JavaScript value into two families:

Family	Stored & copied as	Mutable?	Comparison checks
Primitives	the value itself (**by value**)	No (immutable)	the actual contents
Objects	an address (**by reference**)	Yes (mutable)	the address, not the contents

Every JS value
├── Primitives (by value, immutable)
│   ├── number   → math
│   ├── string   → text
│   ├── boolean   → true / false
│   ├── undefined   → not assigned (engine's emptiness)
│   ├── null   → empty on purpose (your emptiness)
│   ├── bigint   → huge integers
│   └── symbol   → guaranteed-unique id
└── Objects   (by reference, mutable)
    ├── {}   → key → value
    ├── []   → ordered lists
    ├── function   → reusable code
    └── ...everything that isn't a primitive

In-Depth Lecture Notes: JavaScript Operators

1. Introduction: What is an Operator?

In JavaScript, an **operator** is a special symbol or keyword used to perform an operation on values. The values that the operator works on are called **operands**. The combination of operators and operands forms an **expression**, which evaluates to a single value.

// In this expression: 5 + 10
// → `+` is the operator.
// → `5` and `10` are the operands.
// → The entire expression `5 + 10` evaluates to the value `15`.
let result = 5 + 10;

2. Assignment Operators

These operators are used to assign a value to a variable.

= (Assignment): The fundamental assignment operator. It assigns the value on its right to the variable on its left. code JavaScript

let score = 100;

Compound Assignment (Shorthands): These combine a mathematical operation with an assignment. They are very common and make code more concise.

Operator	Example	Equivalent To
+=	x += y	x = x + y
-=	x -= y	x = x - y
*=	x *= y	x = x * y
/=	x /= y	x = x / y
%=	x %= y	x = x % y
**=	x **= y	x = x ** y

let level = 10;
level += 5; // level is now 15
level *= 2; // level is now 30

3. Arithmetic Operators

Used for standard mathematical calculations.

+ (Addition)

(Subtraction)

(Multiplication)

/ (Division)

- (Exponentiation - ES2016): 2 ** 3 evaluates to 8.

% (Remainder / Modulo): Returns the remainder of a division. This is extremely useful for tasks like checking if a number is even or odd. code JavaScript

console.log(10 % 3); // 1 (10 divided by 3 is 3, with a remainder of 1)
console.log(10 % 2); // 0 (An even number always has a remainder of 0 when divided by 2)

++ (Increment) & -- (Decrement): Increases or decreases a number by 1. code JavaScript

Gotcha: Prefix vs. Postfix. The position of the operator matters critically.

Postfix (variable++): The expression evaluates to the variable's **original value**, and *then* the variable is incremented.

Prefix (++variable): The variable is incremented **first**, and *then* the expression evaluates to the **new value**.

```
let postfix = 5;
let prefix = 5;

console.log(postfix++); // Prints 5. `postfix` is now 6.
console.log(postfix);   // Prints 6.

console.log(++prefix); // Prints 6. `prefix` is now 6.
console.log(prefix);   // Prints 6.
```

4. Comparison Operators

These operators compare two values and evaluate to a boolean (true or false).

> (Greater than)

< (Less than)

>= (Greater than or equal to)

<= (Less than or equal to)

The Most Critical Concept: Strict vs. Loose Equality

== (Loose Equality): Compares two values for equality **after** performing type coercion (automatic type conversion). **AVOID THIS OPERATOR.** It has complex rules and can lead to very confusing bugs. code JavaScript

```
console.log(7 == "7");   // true (string "7" is coerced to number 7)
console.log(0 == false); // true (boolean false is coerced to number 0)
```

=== (Strict Equality): Compares two values for equality **without** performing type coercion. It checks if both the **value AND the type** are identical. **ALWAYS PREFER THIS OPERATOR.** code JavaScript

```
console.log(7 === "7");   // false (number is not a string)
console.log(0 === false); // false (number is not a boolean)
```

!= (Loose Inequality) & !== (Strict Inequality): The same rules apply. Always prefer !==.

5. Logical Operators

Used to combine boolean expressions, typically in if statements.

&& (Logical AND): Evaluates to true only if **both** operands are true (or "truthy"). code JavaScript

Short-Circuiting: If the left-hand operand is false, the right-hand operand is **never evaluated**. This is a powerful feature for preventing errors.

```
let user = null;
// This is safe. The code stops at `user` and never tries to access `user.name`.
if (user && user.name === "Admin") { /* ... */ }
```

|| (Logical OR): Evaluates to true if **either** operand is true. code JavaScript

Short-Circuiting: If the left-hand operand is true, the right-hand operand is **never evaluated**. This is commonly used to provide default values.

```
let username = ""; // an empty string is "falsy"
let displayName = username || "Guest"; // displayName becomes "Guest"
```

! (Logical NOT): Inverts the boolean value of its operand. It coerces the operand to a boolean first, then flips it. code JavaScript

```
let isLoggedIn = false;
if (!isLoggedIn) {
  console.log("Please log in.");
}
```

Important Note: "Truthy" and "Falsy"

When used in a logical context, every value in JavaScript has an inherent boolean value.

The 6 Falsy Values: false, 0, "" (empty string), null, undefined, NaN.

Everything else is Truthy, including "false", [] (an empty array), and {} (an empty object).

6. Bitwise Operators

These are less common in web development but are important to know. They operate on the binary (base-2) representation of numbers. **Do not confuse & with && or | with ||.**

Operator	Name	Description	Example (5 is 101, 3 is 011)
&	AND	Sets each bit to 1 if both bits are 1.	5 & 3 → 1 (001)
`	`	OR	Sets each bit to 1 if one of two bits is 1.
^	XOR	Sets each bit to 1 if only one of two bits is 1.	5 ^ 3 → 6 (110)
~	NOT	Inverts all the bits.	~5 → -6
<<	Left Shift	Shifts bits to the left, padding with zeros.	5 << 1 → 10 (1010)
>>	Right Shift	Shifts bits to the right.	5 >> 1 → 2 (010)

7. Other Important Operators

typeof (Unary Operator): Returns a string indicating the type of an operand. code JavaScript

```
console.log(typeof 42);       // "number"
console.log(typeof "hello");  // "string"
console.log(typeof null);     // "object" (This is a famous, long-standing bug)
```

Ternary Operator (? :): A compact shorthand for an if...else statement. It's the only operator that takes three operands. code JavaScript

Syntax: condition ? expressionIfTrue : expressionIfFalse

```
let age = 20;
let message = (age >= 18) ? "You can vote." : "You cannot vote yet.";
console.log(message); // "You can vote."
```

8. Operator Precedence

This determines the order in which operators are executed. For example, multiplication (*) has a higher precedence than addition (+).

```
let result = 2 + 3 * 5; // result is 17, not 25
```

Best Practice: Don't memorize the entire precedence table. When in doubt, use parentheses () to make the order explicit and your code more readable.

```
let result = (2 + 3) * 5; // result is 25. The intent is perfectly clear.
```

How number are stored in Javascript

Why 0.+0.2 = 0.30000000000000004

The computer thinks in Base 2 (binary), but we are giving it a problem in Base 10 (decimal). The translation is not perfect.

Imagine you have a calculator that can only store **10 binary places** after the decimal point.

Step 1: Translate 0.1 (Decimal) to Binary

We'll use our "multiply by 2" method to find the first 10 binary places for 0.1.

0.1 * 2 = 0.2 → **0**

0.2 * 2 = 0.4 → **0**

0.4 * 2 = 0.8 → **0**

0.8 * 2 = 1.6 → **1**

0.6 * 2 = 1.2 → **1**

0.2 * 2 = 0.4 → **0** (The pattern starts repeating here)

0.4 * 2 = 0.8 → **0**

0.8 * 2 = 1.6 → **1**

0.6 * 2 = 1.2 → **1**

0.2 * 2 = 0.4 → **0**

Because our calculator can only store 10 places, it has to stop here.

0.1 (Decimal) gets stored as 0.0001100110 (Binary)

Now, what is the *actual* decimal value of this stored binary number? Let's add up the place values (1/16 + 1/32 + 1/256 ...):

0.0001100110 (Binary) = **0.10009765625** (Decimal)

As you can see, a tiny **rounding error** has already occurred. Our calculator is storing a number that is slightly *larger* than 0.1.

Step 2: Translate 0.2 (Decimal) to Binary

Now we do the same for 0.2.

0.2 * 2 = 0.4 → **0**

0.4 * 2 = 0.8 → **0**

0.8 * 2 = 1.6 → **1**

0.6 * 2 = 1.2 → **1**

0.2 * 2 = 0.4 → **0** (The pattern repeats)

0.4 * 2 = 0.8 → **0**

0.8 * 2 = 1.6 → **1**

0.6 * 2 = 1.2 → **1**

0.2 * 2 = 0.4 → **0**

0.4 * 2 = 0.8 → **0**

0.2 (Decimal) gets stored as 0.0011001100 (Binary)

What is the decimal value of this stored binary?

0.0011001100 (Binary) = **0.19921875** (Decimal)

Again, a tiny **rounding error**. Our calculator is storing a number that is slightly *smaller* than 0.2.

Step 3: The Sum (0.1 + 0.2)

The computer doesn't see 0.1 + 0.2. It sees the **approximations** it has stored.

0.10009765625 (the stored version of 0.1)

0.19921875 (the stored version of 0.2)

0.29931640625

This is the actual result of the sum.

Step 4: The Comparison with 0.3

Now, the user expects the result to be 0.3. But what does the calculator store for 0.3?

Let's convert 0.3 to 10 binary places:

0.3 * 2 = 0.6 → **0**

0.6 * 2 = 1.2 → **1**

0.2 * 2 = 0.4 → **0**

0.4 * 2 = 0.8 → **0**

0.8 * 2 = 1.6 → **1**

0.6 * 2 = 1.2 → **1** (The pattern repeats)

0.2 * 2 = 0.4 → **0**

0.4 * 2 = 0.8 → **0**

0.8 * 2 = 1.6 → **1**

0.6 * 2 = 1.2 → **1**

0.3 (Decimal) gets stored as 0.0100110011 (Binary)

What is the decimal value of this stored binary?

0.0100110011 (Binary) = **0.2998046875** (Decimal)

Conclusion: Why the Inaccuracy Occurs

Now we can show the user the final comparison the computer makes:

Is 0.29931640625 (the sum) equal to 0.2998046875 (the stored 0.3)?

No. They are different.

The inaccuracy occurs because:

Imperfect Translation: The numbers 0.1, 0.2, and 0.3 cannot be perfectly represented in binary.

Rounding Errors: The computer must cut off the infinite binary sequence, which creates tiny rounding errors for each number.

Accumulated Errors: The tiny, individual errors from 0.1 and 0.2 add up to a final result that is different from the tiny error produced when storing 0.3 directly.

Conversion in JS

Converting data from one type to another is a fundamental task in JavaScript. This process is called **Type Conversion** or **Type Casting**.

There are two main ways this happens in JavaScript:

Implicit Conversion (Coercion): JavaScript does it automatically behind the scenes. This is what happens with the == operator. It can be convenient but is often a source of bugs because it's not obvious.

Explicit Conversion: You, the developer, intentionally write code to convert a value from one type to another. **This is the recommended, safe, and professional way to handle type conversions.**

Let's focus on **Explicit Conversion**.

The "First Thought" Principle

The easiest way to think about explicit conversion is to use the **name of the type you want as a function.**

Want a **Number**? Use Number().

Want a **String**? Use String().

Want a **Boolean**? Use Boolean().

This simple pattern is the most common and readable way to convert types.

Converting to a String

This is the simplest conversion. Almost any value can be represented as a string.

Method 1: The String() Function (Recommended)

This is the safest and most reliable method. It works for any value, including null and undefined.

```
let num = 123;
let strNum = String(num); // "123"

let bool = true;
let strBool = String(bool); // "true"

let value = null;
let strNull = String(value); // "null"

let arr = [1, 2];
let strArr = String(arr); // "1,2"
```

Converting to a Number

This is a very common task, especially when getting user input from the web, which is always a string.

Method 1: The Number() Function (Recommended)

This is the most direct way. It follows a clear set of rules:

"123" → 123

" 123 " (with spaces) → 123

"123.45" → 123.45

true → 1

false → 0

null → 0

undefined → NaN

"hello" (non-numeric string) → NaN

```
let str = "99.5";
let num = Number(str); // 99.5

let strSpaces = "   100   ";
let numSpaces = Number(strSpaces); // 100

let invalidStr = "apple";
let notANumber = Number(invalidStr); // NaN
```

Method 2: parseInt() and parseFloat()

These functions are more specific. They parse a string from left to right and stop when they hit a non-numeric character.

parseInt(): Parses for an integer.

parseFloat(): Parses for a floating-point number.

```
parseInt("100px"); // 100 (it stops at 'p')
parseFloat("3.14em"); // 3.14 (it stops at 'e')

parseInt("Chapter 2"); // NaN (because it doesn't start with a number)

Number("100px"); // NaN (because the whole string is not a valid number)
```

parseInt() is often useful for extracting a number from the beginning of a string.

Method 3: The Unary Plus + Operator (A common trick)

Placing a + in front of a value is a concise way to convert it to a number, following the same rules as the Number() function.

```
let str = "50";
let num = +str; // 50 (as a number)
```

Converting to a Boolean

In JavaScript, every value has an inherent "truthiness." Converting to a boolean makes this explicit.

Method 1: The Boolean() Function (Recommended)

This is the clearest way. The rules are simple: there is a small, specific list of "falsy" values. **Everything else is "truthy."**

The 6 Falsy Values:

false

0 (the number zero)

"" (an empty string)

null

undefined

NaN

```
Boolean(0);        // false
Boolean("");       // false
Boolean(null);     // false
Boolean(undefined); // false
Boolean(NaN);      // false

// Everything else is truthy!
Boolean(100);       // true
Boolean("hello");   // true
Boolean("false");   // true (a non-empty string is truthy)
Boolean([]);        // true (an empty array is an object, and is truthy)
Boolean({});        // true (an empty object is truthy)
```

How to compare value

Part 1: The Strict Equality Operator (===, !==)

This one is simple because it has only one rule.

Rule 1: Check the Types.

If the types are **different**, the result is false. No coercion happens.

If the types are the **same**, compare the values:

For primitives (number, string, boolean), the values are equal if they are the same.

For null and undefined, they are only equal to themselves.

For objects (including arrays), the references must point to the **exact same object in memory**.

Special Case: NaN === NaN is always false.

Part 2: The Loose Equality Operator (==, !=)

This is the **Abstract Equality Comparison Algorithm**. The engine checks these rules in order.

Rule 1: Same Types.

If the operands have the **same type**, behave exactly like the strict equality operator (===).

Rule 2: null and undefined Special Case.

If one operand is null and the other is undefined, the result is true.

Rule 3: String and Number.

If one operand is a string and the other is a number, convert the string to a number and re-compare.

Rule 4: Boolean Conversion.

If one operand is a boolean, convert the boolean to a number (true → 1, false → 0) and re-compare.

Rule 5: Object and Primitive.

If one operand is an object and the other is a string, number, or symbol, convert the object to a primitive (by calling valueOf() then toString()) and re-compare.

Rule 6: Default.

If none of the above rules apply, the result is false.

Part 3: The Relational Operators (<, >, <=, >=)

This is the **Abstract Relational Comparison Algorithm**. It has a different, simpler (but still tricky) set of rules. The engine's first step is to get a primitive value from both operands.

Step 1: Convert to Primitives (if necessary).

If an operand is an object, convert it to a primitive by calling its valueOf() and then toString() methods. The rest of the rules will operate on these new primitive values.

Rule 1: String vs. String.

If **both** operands are strings (after the initial conversion step), perform a **lexicographical (dictionary) comparison** character by character. Do **not** convert them to numbers.

Rule 2: The Default Numeric Conversion.

In **all other cases**, convert **both** operands to numbers and perform a numeric comparison.

Coercion Details for this rule:

strings are parsed ("5" → 5, "hello" → NaN).

booleans are converted (true → 1, false → 0).

null is converted to 0.

undefined is converted to NaN.

Special Case: If either of the final numbers is NaN, the result of the comparison is **always false**.

A Simple Decision Tree for Comparisons

Here is a mental checklist you can follow when you see a comparison:

1. Is the operator === or !==?

YES: No coercion. Are the type and value identical? Done.

2. Is the operator == or !=?

Are the types the same? → Compare values.

Is it null == undefined? → true.

Is a boolean involved? → Convert boolean to number, then restart the check.

Is a string and number involved? → Convert string to number, then compare.

Is an object involved? → Convert object to primitive, then restart the check.

3. Is the operator <, >, <=, or >=?

Are **both** sides strings? → Use dictionary rules.

NO? → Convert **both** sides to numbers (null→0, undefined→NaN) and do a numeric comparison. If NaN appears, the result is false.

This ordered breakdown reveals why the behavior is so complex: there are three distinct sets of rules, and the rules for loose equality (==) are very different from the rules for relational comparisons (>=).